Databricks Feature Engineering in Unity Catalog

In this chapter, we will focus on several types of features. Feature types can be roughly grouped based on when they are calculated relative to the time of model prediction. The three types we cover in this chapter are batch, streaming, and on-demand:

- **Batch features**: These are features that are generated hours, days, or even further ahead of when the feature will be used as a model input. There are several reasons for this; the model may be offline and run in a batch prediction fashion, or perhaps the feature values change slowly relative to the model inference timeline. In these cases, it is not necessary to recompute the values more frequently. An example of a batch feature is the `holidays` field in our *Favorita Sales Forecasting* project, as we can compute it before we need it and don't expect the values to change frequently.
- **Streaming features**: These are features processed in real or near-real time as the source data is ingested in the data pipeline, allowing for continuous and incremental feature creation. To demonstrate this, we will calculate a streaming feature for our *Streaming Transactions* project with Spark Structured Streaming.
- **On-demand features**: Unlike batch or streaming features, on-demand features are only computed when needed, which is to say, at the time of inference. These features are crucial for scenarios where feature values are unknown beforehand and must be calculated in real time. We will delve into the mechanics of computing and storing these features, demonstrating their implementation and integration into predictive models.

Any Unity Catalog table with a defined primary key constraint can be a centralized repository for materialized, pre-computed (e.g., batch or streaming) features. These types of tables, whose explicit purpose is to centrally store features to be used throughout an organization's analytics, data science, and machine learning projects, are commonly called **feature tables**. Feature tables allow data scientists to share their engineered features and find features that other team members have built.

They are a wonderful way to ensure business logic is centrally stored, save team members from recreating already-created features, and prevent duplicate work. Plus, Unity Catalog manages all data's discoverability, lineage, and governance, so your feature tables are easily managed just like any other table. In Databricks, any table with a non-nullable primary key can be a feature table. While there is no specific API call to list all tables eligible to be a feature table or to list those officially created as a feature table, we recommend using tags or naming conventions to identify feature tables. This book's example projects will follow the convention of appending `_ft` at the end of feature table names. Feature Engineering in Unity Catalog provides the `FeatureEngineeringClient` Python client (commonly imported under the alias of `fe`) for creating and updating feature tables (*Figure 5.1*):